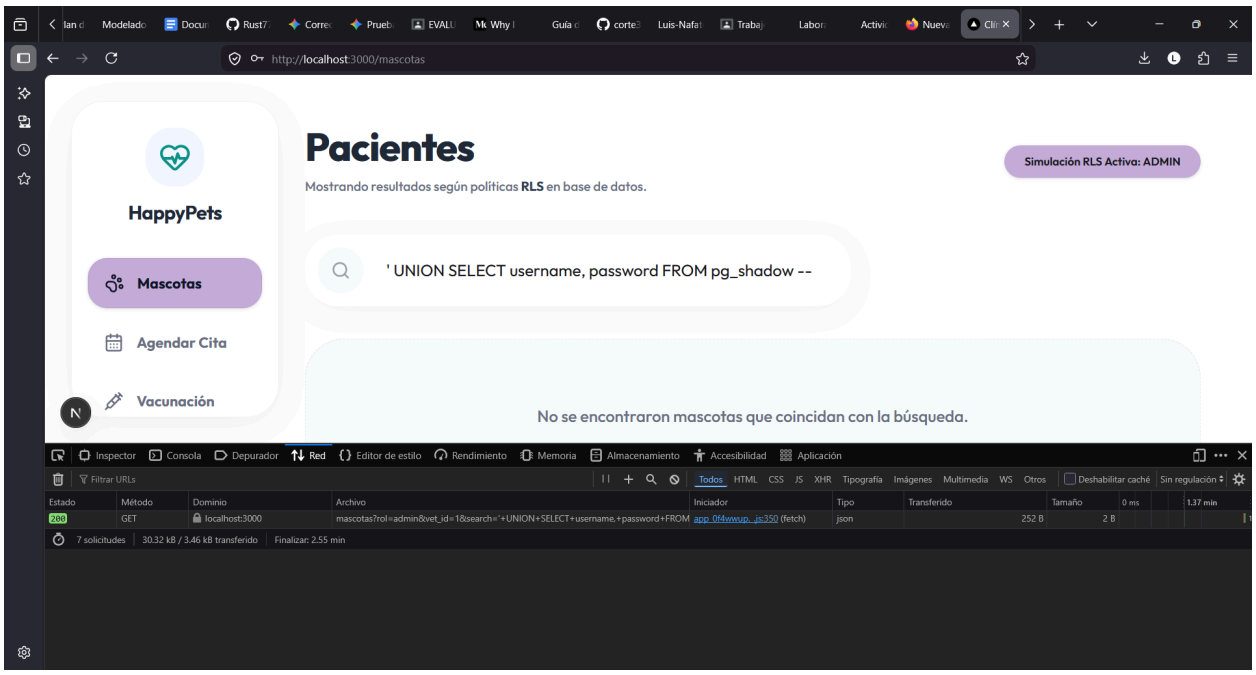
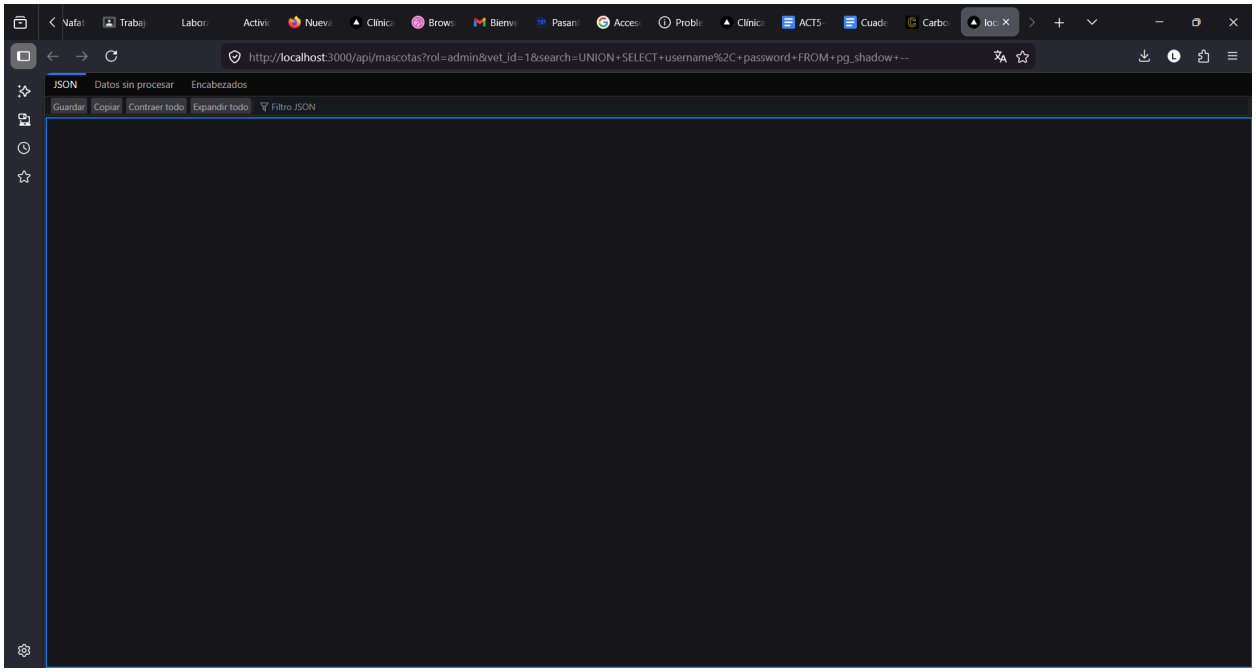
Sección 1: Tres ataques de SQL injection que fallan

A continuación, tres ataques dirigidos al endpoint de búsqueda de pacientes.

Ataque 1: Union-based (Extracción de credenciales del motor)

- Input exacto probado: ' UNION SELECT username, password FROM pg_shadow --
- Pantalla del frontend: Campo principal de búsqueda de pacientes.





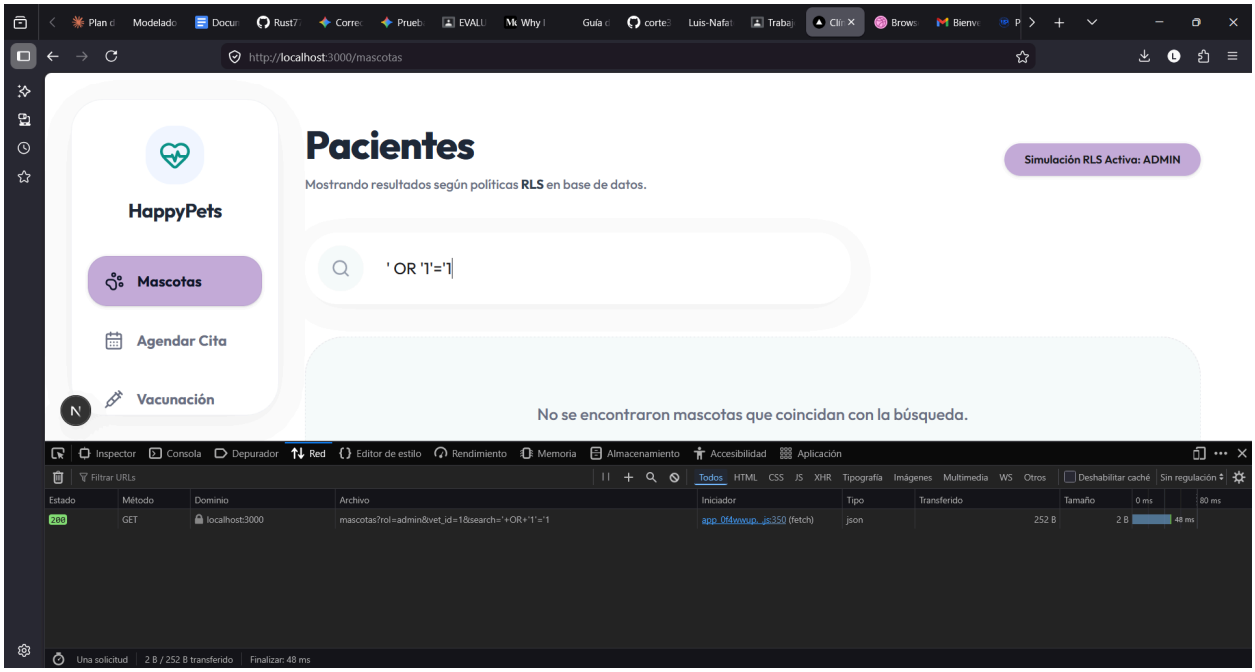
- **Prueba de que el ataque falló: El backend no acopló la consulta maliciosa. Tratando el payload de forma literal, buscó un paciente con ese nombre extraño. La base de datos respondió pacíficamente con un arreglo vacío [ ]. No hubo fuga de datos.**

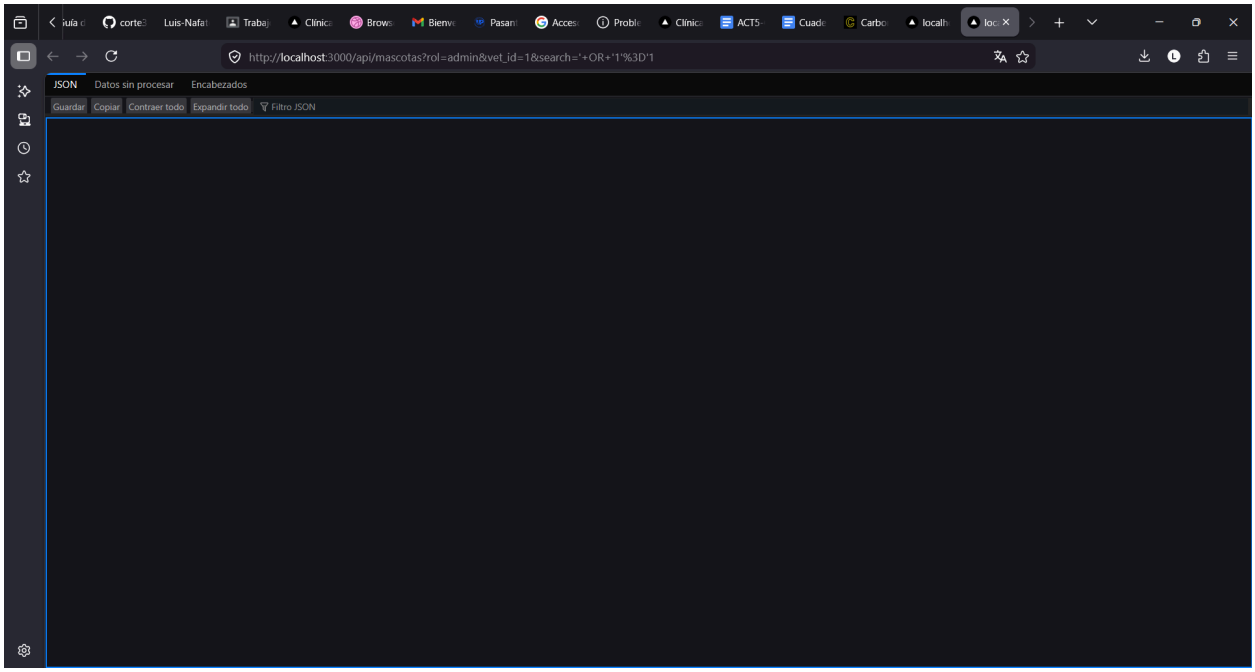
Línea exacta de defensa:

javascript const query = ... WHERE m.nombre ILIKE $1 OR m.especie ILIKE $1 ...; const params = [%${searchTerm}%]; await client.query(query, params);

Ataque 2: Quote-escape clásico (Bypass lógico)

- Input exacto probado: ' OR '1'='1
- Pantalla del frontend: Campo principal de búsqueda de pacientes.
- **Prueba de que el ataque falló:** El sistema no devolvió el listado completo de todas las mascotas. La interfaz mostró "0 resultados", confirmando que la base de datos evaluó el input como un string literal y no como una cláusula condicional verdadera.

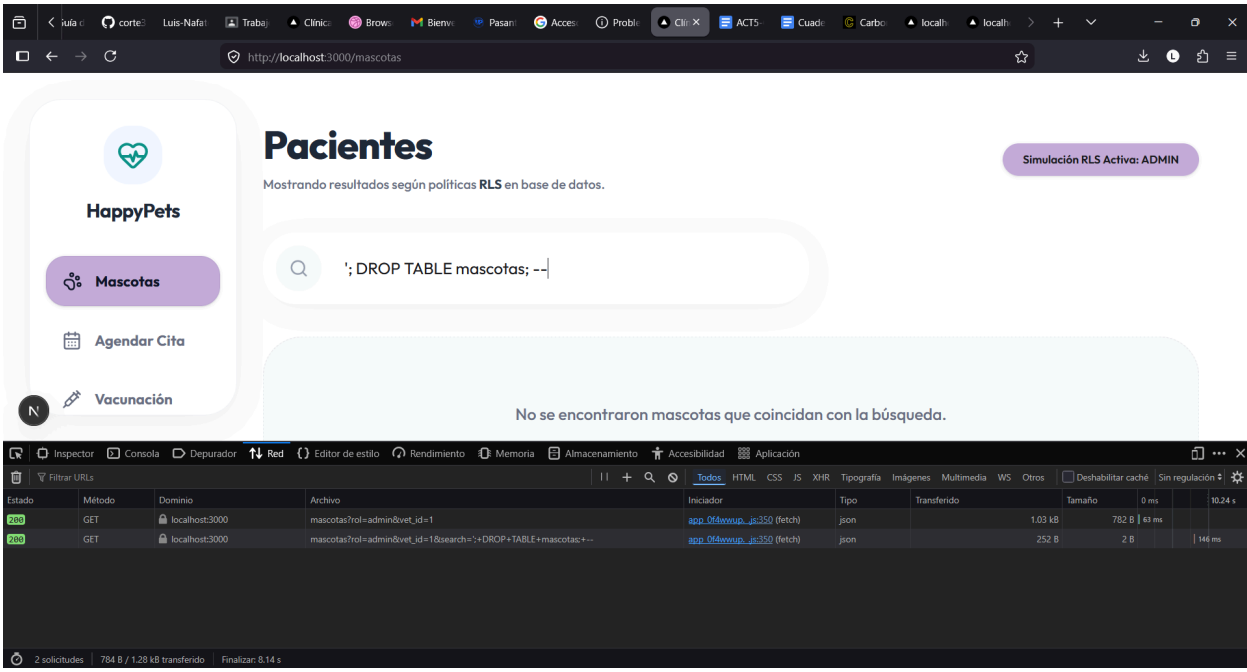


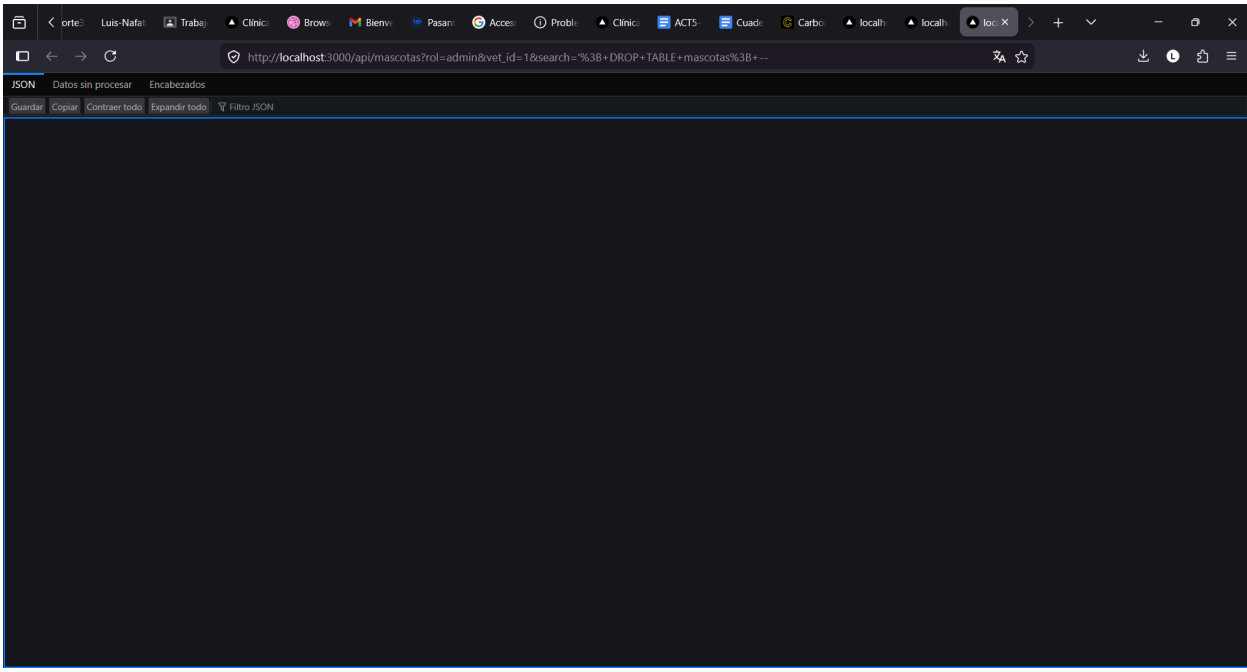


- **Línea exacta de defensa: Defendido por las mismas líneas exactas de Prepared Statements mencionadas en el Ataque 1.**

Ataque 3: Stacked query (Intento de destrucción)

- Input exacto probado: '; DROP TABLE mascotas; --
- Pantalla del frontend: Campo principal de búsqueda de pacientes.
- Prueba de que el ataque falló: La base de datos no cerró la instrucción SELECT ni ejecutó el comando DROP. La tabla de mascotas permaneció intacta y el buscador devolvió [].





- **Línea exacta de defensa: Defendido por las mismas líneas exactas de Prepared Statements mencionadas en el Ataque 1.**

Sección 2: Demostración de RLS en acción

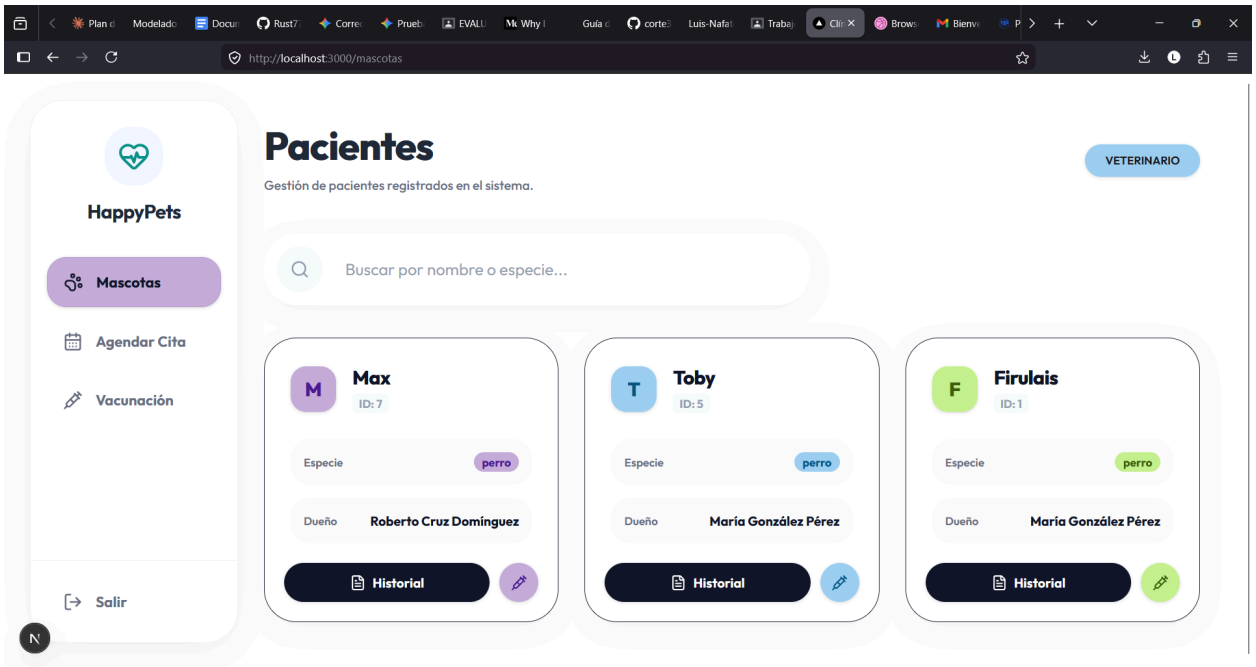
El sistema implementa políticas Row Level Security (RLS) en PostgreSQL, garantizando la segregación estricta de pacientes entre veterinarios.

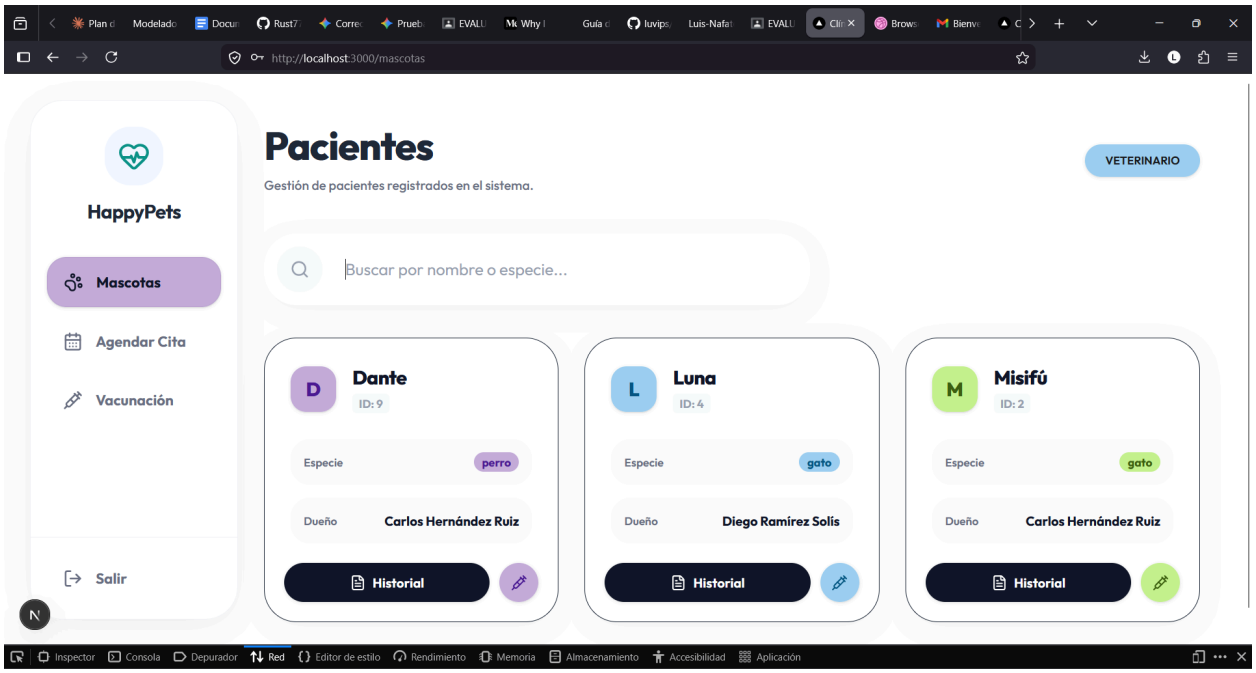
Setup de la prueba:

- Veterinario Autorizado: Dr. López (vet_id = 1). Mascotas que le pertenecen: Firulais (1), Toby (5) y Max (7).
- Objetivo Malicioso: Interceptar la petición e intentar acceder a la mascota ID=9 (que le pertenece a otro doctor).

Demostración del comportamiento:

1. Acceso legítimo (Dr. López): Al hacer una petición para cargar sus mascotas, el backend fuerza el contexto en la BD (SET LOCAL ROLE rol_veterinario; SET LOCAL app.current_vet_id = '1';). El RLS filtra y retorna solo las filas permitidas: Firulais, Toby y Max.





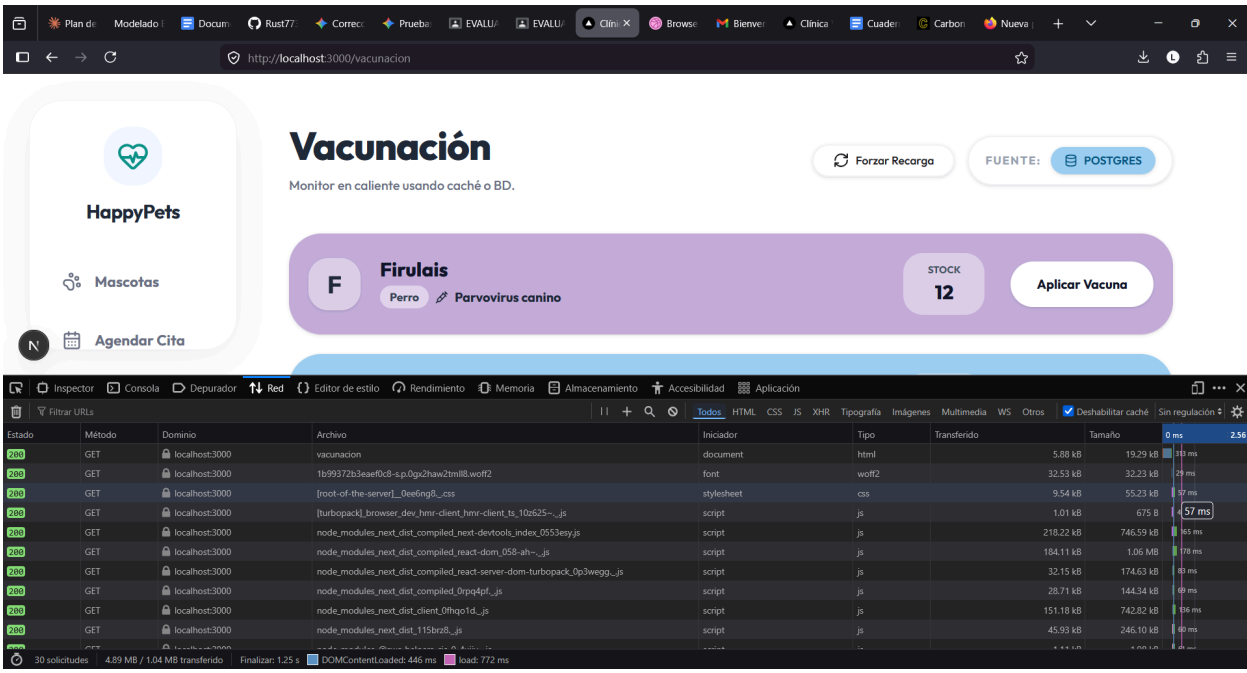
Frase explicando la política: Este comportamiento ocurre porque la política RLS en la tabla mascotas verifica en tiempo de ejecución que el dueño del registro coincida con la variable de entorno de sesión del motor: vet_id = current_setting('app.current_vet_id').

Sección 3: Demostración de caché Redis funcionando

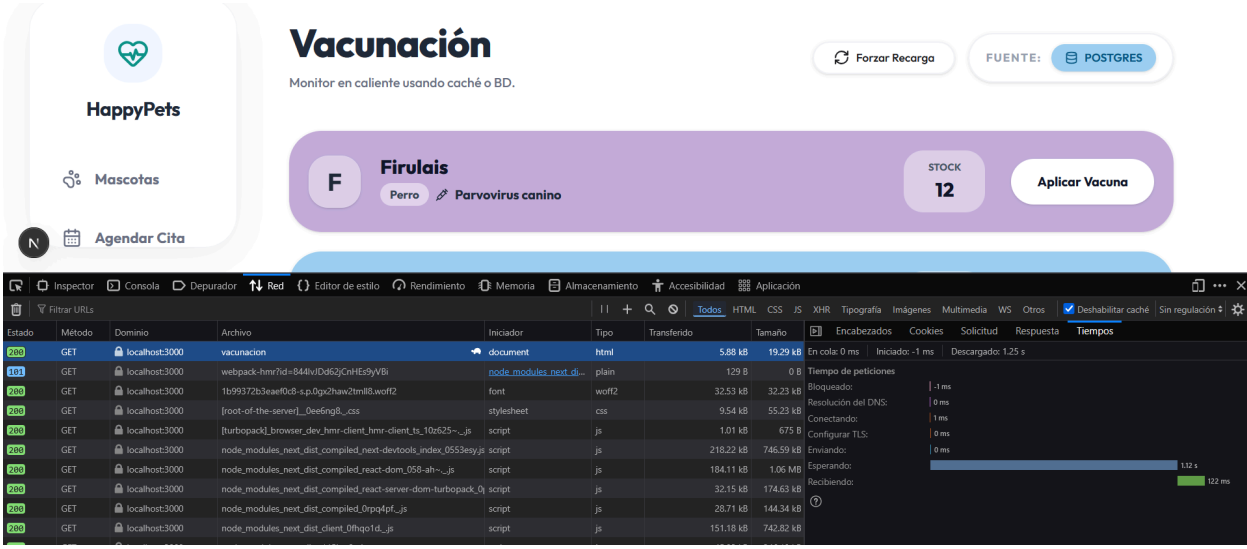
Se implementó el patrón Cache-Aside usando Redis para mitigar cargas concurrentes sobre la vista v_mascotas_vacunacion_pendiente.

Logs y Latencias:

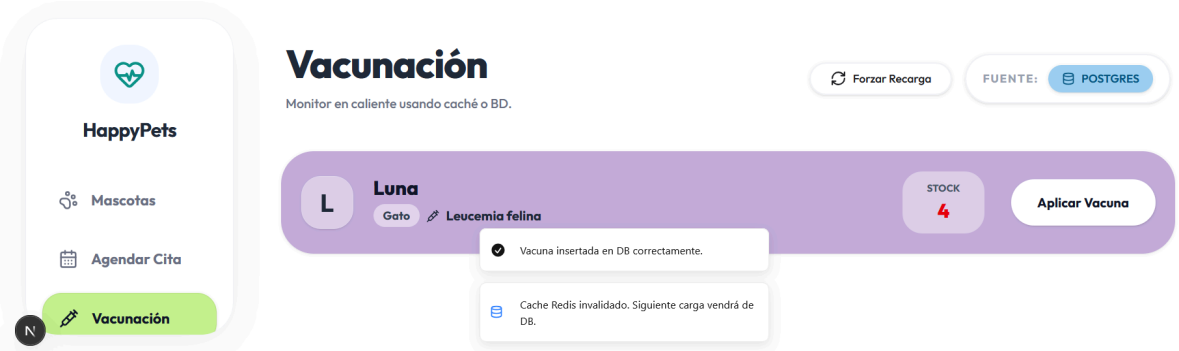
1. Primera consulta a vacunación pendiente (Cache MISS):
    - El backend intenta leer redis.get('vacunacion_pendientes_cache') y no encuentra nada.
    - Consulta PostgreSQL. Latencia típica de motor: ~500ms.



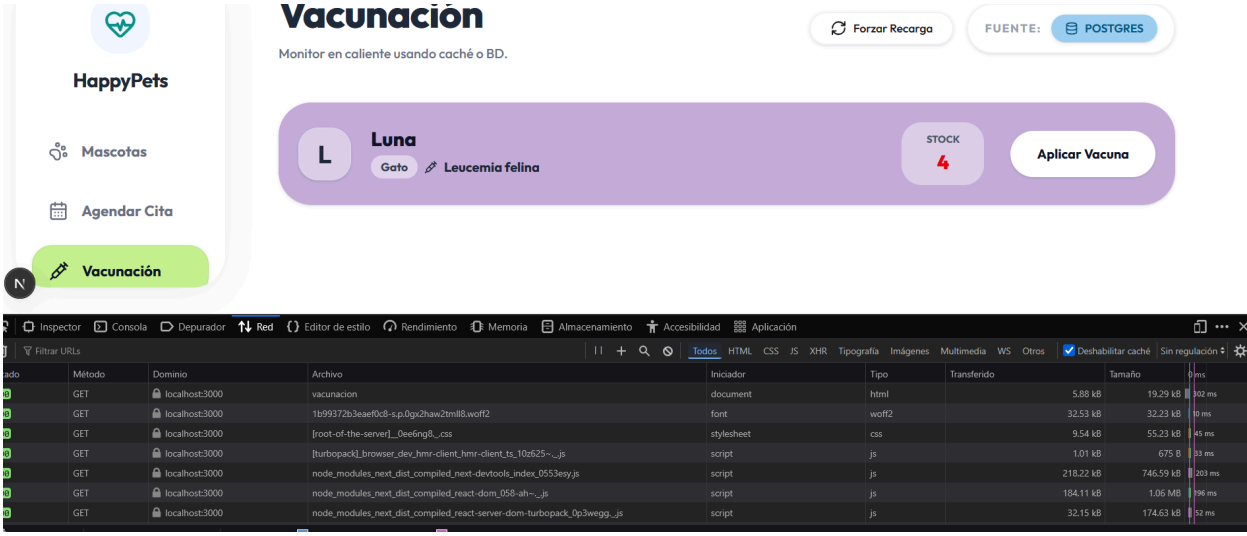
2. Segunda consulta inmediata (Cache HIT):
    - El backend encuentra los datos en RAM. La base de datos no recibe tráfico. Latencia de Redis: <10ms.



3. POST de aplicación de vacuna (Invalidación):
    - Se registra una nueva vacuna en POST /api/vacunas. El sistema ejecuta la limpieza para evitar Stale Data.



4. Tercera consulta después de invalidación (Cache MISS):
    - Al recargar la vista de pendientes, la caché ya no existe. El sistema va a la BD, demora los ~500ms iniciales y trae los datos frescos con la vacuna ya aplicada.



Detalles técnicos de la estrategia:

- Key utilizada: vacunacion_pendientes_cache
- TTL elegido: 300 segundos.
- Por qué: Se eligió un TTL de 5 minutos porque v_mascotas_vacunacion_pendiente es una consulta de alto costo computacional. Si la invalidación manual (redis.del) llegara a fallar durante much estrés, la caché se autodestruirá en 5 minutos. Esto garantiza que el sistema sobreviva a inyecciones de miles de peticiones sin colapsar la BD, manteniendo el desajuste de datos en un límite máximo aceptable de 5 minutos.